

From Prompt Engineering to Context Engineering

Christopher Teoh

Outline

- Why AI Fails
- Prompts to Context Systems
- How AI Works
- ICCO: Designing Inputs That Work
- From Text to Structured Outputs
- Skills, Systems & Testing



Top Large Language Models as of 2026



Open AI



Claude



Gemini



Qwen



DeepSeek



Grok

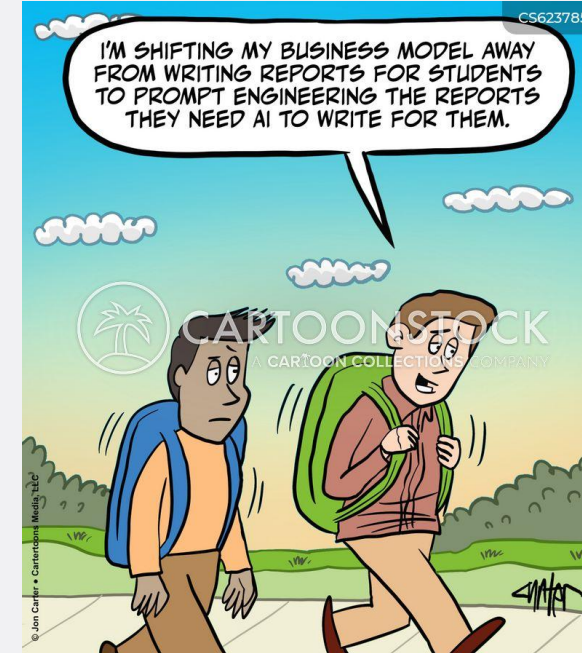


Top Large Language Models as of 2026

Comparison Takeaway

- **GPT-5:** Most consistent **all-rounder** (coding, reasoning, long context)
- **Claude (Opus / Sonnet):** Best for **deep analysis** and long-form reasoning
- **Gemini Flash:** **Fast**, cost-efficient for real-time and high-volume use
- **DeepSeek / Qwen 3:** **Strong reasoning + multilingual** at lower cost
- **Llama 4:** Open-weight **flexibility** for self-hosting and control





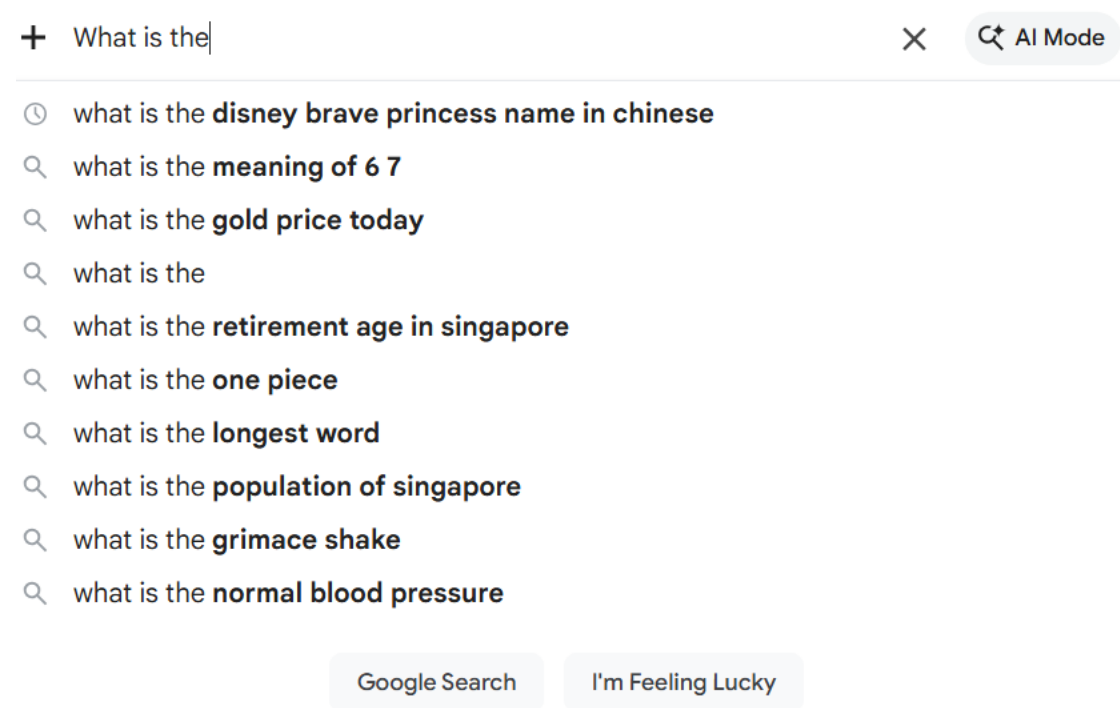
Most AI failures are NOT caused by poor prompts; they are caused by **poorly designed context.**

<https://www.cartoonstock.com/cartoon?searchID=CS621607>
<https://www.cartoonstock.com/cartoon?searchID=CS623785>



Generative AI Fundamentals

- Generative AI models do not “think” like humans.
 - They have **no intent, understanding, or awareness**.
- These models predict the **most likely sequence of tokens** based on patterns learned during training.
 - The quality of the output is entirely dependent on the quality of the input.
- The model does not know an organization’s:
 - Business goals
 - Target audience
 - Expectations



Example from Google Search





Prompt & Context Engineering



Introduction

- Prompt engineering looks like a simple skill: We type instructions into an AI model, and it produces an output (i.e. summaries, ideas, code) instantly.
- This excitement turns into frustration when outputs become **inconsistent** or **misaligned** with expectations.
- **Note:** The transition from prompt engineering to context engineering represents a shift from **interacting** to **designing** how AI behaves systematically.

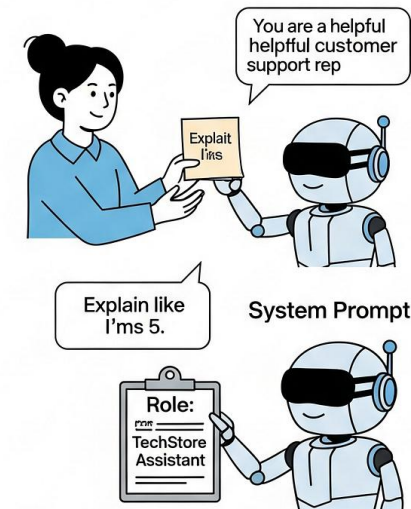


From Prompt Engineering to Context Engineering

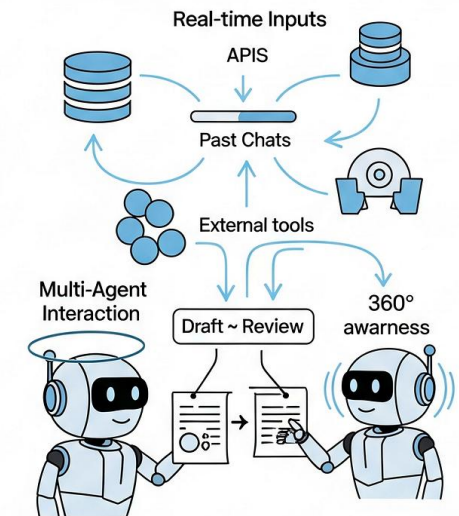
- Prompt engineering focuses on crafting **individual instructions**, while context engineering expands the scope to include **everything that shapes** the AI's response:
 - Background information
 - Constraints
 - Structure
 - Reusable systems
- In real-world, this difference determines whether AI is a:
 - Toy for experimentation
 - Tool for automation, and decision-making

Prompt Engineering VS Context Engineering

Teaching AI what to do



Giving AI the bigger picture



Prompt

- A prompt is the input instruction given to the AI. It defines what the model is supposed to do.
- For example:
 - “Summarise this document”
 - “Write a marketing email”
- **Problem:** Incomplete because they **lack** *specificity*, *context*, and *structure*.



LLM Mechanics

To peek under the hood, we need to understand how AI processes information.

- The AI receives **input** (prompt + context)
- It **processes** the input within its context window
- It **generates** an output based on learned patterns



All these are
processed
together



Solution: The ICCO Framework

Instruction, Context, Constraints, Output

- Each component plays a distinct role:
 - **Instruction** defines the task
 - **Context** provides background
 - **Constraints** guide behaviour
 - **Output format** structures the result
- This framework mirrors how AI processes inputs; when all four components are well designed, outputs become far more reliable and useful.



ICCO - Instruction

- Instruction is the most visible part of a prompt and tells the AI what action to perform. Many users underestimate how important clarity is at this stage.

Vague	Specific
“Write something about AI”	“Write a beginner-friendly explanation of AI for business professionals with no technical background.”
The prompt is too generic	This version works better because it narrows the scope and aligns the output with a specific audience.

- Instruction is the **destination** in a journey. If the destination is unclear, the AI cannot navigate effectively.



ICCO - Context

- Context is the most important (and most overlooked) component of effective prompting.
 - It influences which **token patterns** the AI retrieves during generation. Without it, the AI is essentially operating in a vacuum.
- It refers to all the information the AI needs in order to perform the task accurately such as:
 - **Goal**
 - **Target Audience**
 - **Real-world Constraints**
- Poor results are often caused by **missing context**, not bad prompts.



A Note on Context Window

API PROVIDER ↕	MODEL ↕	CONTEXT ↕	MODEL QUALITY ↕	PRICE ↕	OUTPUT TOKENS/S ↕	LATENCY ↕	FURTHER ANALYSIS
		CONTEXT WINDOW ↕	INDEX Normalized avg ↕	BLENDED USD/1M Tokens ↕	MEDIAN Tokens/s ↕	MEDIAN First Chunk (s) ↕	
OpenAI	GPT-4o	128k	100	\$7.50	83.8	0.45	Model Providers
OpenAI	GPT-4 Turbo	128k	94	\$15.00	27.9	0.68	Model Providers
Microsoft Azure	GPT-4 Turbo	128k	94	\$15.00	32.3	0.57	Model Providers
OpenAI	GPT-4o mini	128k	85	\$0.26	166.1	0.57	Model Providers
OpenAI	GPT-4	8k	84	\$37.50	25.9	0.83	Model Providers
Microsoft Azure	GPT-4	8k	84	\$37.50	22.9	0.57	Model Providers
OpenAI	GPT-3.5 Turbo Instruct	4k	60	\$1.63	71.1	0.36	Model Providers
Microsoft Azure	GPT-3.5 Turbo Instruct	4k	60	\$1.63	136.9	0.61	Model Providers
OpenAI	GPT-3.5 Turbo	16k	59	\$0.75	84.9	0.45	Model Providers
Microsoft Azure	GPT-3.5 Turbo	16k	59	\$0.75	67.3	0.34	Model Providers
Google	Gemini 1.5 Pro	2m	95	\$5.25	58.3	1.03	Model Providers
Google	Gemini 1.5 Flash	1m	84	\$0.53	164.9	1.03	Model Providers
deepinfra	Gemma 2 (27B)	8k	78	\$0.27	32.8	0.25	Model Providers

Key Metrics for Comparison

- **Context Window:** Maximum number of combined input, output, contextual information tokens.
- Latency: Time to first token received after the API request is sent.
- Cost: Price per million tokens for both input and output.
- Performance: Normalized average relative performance across Chatbot arena, MMLU, and MT-Bench.
- Output Speed: Tokens per second received while the model generates tokens.

<https://medium.com/@genai.works/comprehensive-comparison-of-large-language-models-llms-0da7e894e419>



Example: Context

- Consider the difference between these prompts:
 - A. “Write a marketing email”
 - B. “Write a marketing email for a cybersecurity course targeting mid-career professionals in Singapore who are worried about job security”
- **Prompt B** produces better output because it **reduces uncertainty**. The AI no longer has to guess the audience or purpose.
- It can focus on crafting a message that aligns with the given situation.



ICCO - Constraint

- Constraints (Guardrails) are **boundaries** that govern how the AI produces its output.
- Without constraints, the AI tends to:
 - be overly verbose
 - include unnecessary details
 - vary in structure
- By adding constraints, we **reduce variability** and **improve consistency**.



Example: Constraint

- Examples of constraints include:
 - Limiting the length of the response
 - Specifying tone (formal, casual, persuasive)
 - Restricting behaviour (e.g. “do not make assumptions”)
- For example: “Limit the summary to 5 bullet points and focus only on key risks and actions.”
- This constraint forces the AI to prioritise important information and behave in a consistent and reliable manner.



ICCO - Output

- Output format determines how the response (output) is **structured**.
- This is often the difference between:
 - Human-readable output (i.e. structured list)
 - System-usable output (i.e. JSON / XML)
- For example, a **free-text summary** is useful for reading, but is difficult to integrate into workflows or systems.
- In contrast, **structured output** (such as JSON) can be directly ingested in applications.



Structured Output

```
{  
  "summary": "",  
  "risks": "",  
  "actions": ""  
}
```

- The JSON format above ensures:
 - consistency across outputs
 - easier parsing by software systems
 - clearer separation of information
- From a practical perspective, output formatting is what transforms AI from a “chat tool” into a **workflow component**.



Example: Structured Output

- Scenario:
 - Imagine you're building an app that uses AI to analyze customer reviews.
 - You ask the AI: “Summarize this review and tell me the product mentioned and the customer's overall sentiment.”
- Output:
 - “Well, it seems like the customer, SA Tan bought the ‘MegaWidget 3000’ and was quite unhappy. He mentioned it broke after just two days and found the whole experience very frustrating.”



Example: Structured Output (Con't)

That works for a human, but not for the app / automation. Our code now has to:

- Figure out the **product name** (“MegaWidget 3000”)
- Determine the **sentiment** (“unhappy” or “frustrating”)
- Extract the **customer name**.
- Always use similar phrasing. If it says “**felt disappointed**” instead of “unhappy,” our sentiment parser **may break**.



Example: Structured Output (Con't)

- The previous approach was not repeatable. Ideally, we want:

```
review_analysis:  
product_name: MegaWidget 3000  
sentiment: Negative  
summary: Customer reported the product broke after two days,  
leading to frustration.
```

- **This is the strength of structured output.** It turns conversational text into clean, organized data that your code can work with.



Activity #1 Job Hunt Intelligence Test

- Duration: 30 minute
- Material:

From Prompt to Context Engineering | 8 May 2026

Activity 1: Job Hunt Intelligence Test

Step 1: Pick a Job Scenario

- Resume writing
- Interview preparation
- Career switching
- Salary negotiation
- Networking / LinkedIn outreach
- Job application strategy

Step 2: Prompt Your AI
Use this: "Create a 10-question quiz for job seekers on [your scenario]."
Include:

- 2 trick questions that sound correct but are actually wrong
- realistic scenarios (not textbook theory)
- an answer key with short explanations

Step 3: Take the Assessment
Note: Don't overthink. Answer like you would in real life, not what you think sounds smart.

Step 4: Audit the AI
Now switch gear from user → evaluator. This is the real learning.
Ask:

- Which answers are actually useful vs generic fluff?
- Did any "correct" answers feel questionable?
- Were the trick questions truly misleading—or just badly written?
- Does this advice work in Singapore / your industry context, or is it too generic?

Step 5: Improve the System
Now refine your prompt using COO (customize below to suit your needs):

- Instructions: "Create a 10-question quiz focused on job search strategies for fresh graduates entering the workforce"
- Context #1: "Target audience: 20-27 years old, recent graduates in Singapore, limited work experience, applying for entry-level roles, concerned about competitiveness and job stability" OR
- Context #2: "Fresh graduates competing in a saturated job market, applying through LinkedIn, job portals, and referrals, with little to no internship experience"
- Constraints: "Keep explanations under 2 sentences. Avoid generic advice. Include 2 trick questions that sound correct but are wrong"
- Output format: "Return in table format: Question | Options | Answer | Explanation"

Step 6: Re-run & Compare
Point for reflection:

- Did the second version feel sharper?
- Less generic?
- More aligned to real-world decisions?

DigiPen Institute of Technology Singapore



Activity #1 Debrief

-  **Wrong facts** (hallucinated)
-  **Wrong audience or context** (not Singapore-relevant)
-  **Wrong tone or style** (too generic, too formal, too fluffy)
-  **Wrong difficulty** (too unrealistic)
-  **Not actionable** (sounds good but useless in real life)



32 ChatGPT Power User Slash Commands: The Ultimate Cheat Sheet



Speed & Efficiency



Higher Quality



Brevity



Predictable Workflow



Instant Mode Switching



SPEED & FORMATTING (Get to the point)

/ELI5: Explain Like I'm 5
(Complex topics simplified)

/TLDL: "Too Long; Didn't Listen"
(Summarize long text)

/BRIEFLY: Ruthless constraint on length
(Quick definitions)

/EXEC SUMMARY: High-level summary for decision-makers

/CHECKLIST: Convert to actionable checkbox list

/FORMAT AS [Type]: Force output format (Table, JSON, Markdown).
Example: **/FORMAT AS TABLE:**
Compare iPhone 15 vs 14 specs.



PERSONA & TONE (Change the voice)

/ACT AS [Role]: Set specific persona
(e.g., **/ACT AS Michelin Chef**)

/TONE [Mood]: Modify emotional weight
(Formal, Sarcastic, Urgent)

/AUDIENCE [Target]: Adapt for specific group
(Experts, Beginners)

/JARGON: Use technical, industry-specific vocabulary

/DEV MODE: Raw, technical developer perspective

/PM MODE: Project Manager persona
(Timelines, Risks, Resources)



DEEP LOGIC & REASONING (Think harder)

/STEP-BY-STEP: Force AI to show work
(Reduces errors)

/CHAIN OF THOUGHT: Connects logic between ideas

/FIRST PRINCIPLES: Break down to fundamental truths

/DELIBERATE THINKING: Slow down reasoning
(Complex strategy)

/NO AUTOPILOT: Avoid generic, lazy answers

/REFLECTIVE MODE: Reflect on own answer for quality

/SYSTEMATIC BIAS CHECK: Scan response for biases

/EVAL-SELF: Critical self-evaluation at end



ANALYSIS & STRATEGY (Big picture thinking)

/SWOT: Strengths, Weaknesses, Opportunities, Threats

/COMPARE: Side-by-side comparison
(Best with **/FORMAT AS TABLE**)

/MULTI-PERSPECTIVE: Explore 3-4 viewpoints
(Economic, Social, Ethical)

/PARALLEL LENSES: Examine issue through specific lenses

/PITFALLS: Focus on what could go wrong/mistakes

/METRICS MODE: Include measurable KPIs/numbers

/CONTEXT STACK: Keep previous context layers active



ADVANCED CONTROL (Power user tools)

/ROLE: TASK: FORMAT:: "God Mode" single-line definition.
Example: **/ROLE:** Teacher

/TASK: Explain Gravity

/FORMAT: Analogy

/SCHEMA: Structured outline or data model

/REWRITE AS [Style]: Transform text
(e.g., **/REWRITE AS Seinfeld script**)

/BEGIN WITH / END WITH:
Constraint sentence start/end

/GUARDRAIL: Set strict negative constraints
("Do not mention X")

3 ALTERNATIVE APPROACHES (Choose your method)

1) "Prompt Spine" Method



Most control. Best for complex tasks.

2) "Modifier Stack" Method



Great for speed and specific style.

3) "God Mode" Single-Line Method

/ROLE: Teacher
/TASK: Explain Gravity
/FORMAT: Analogy

Rapid execution. Combine Role, Task, Format in one.



Prompt Spine

- **Task:** Rewriting a sensitive, direct email to a colleague.
- Instead of just saying "*make this email nicer,*" you layout the exact spine so the AI gives you an instantly usable draft instead of a generic response.

[Role]: You are an empathetic Team Lead.

[Goal]: Rewrite my rough draft email to an employee who missed a deadline, keeping it supportive but firm.

[Constraints]:

- Do not sound accusatory or passive-aggressive.
- Keep the total length under 150 words.
- Explicitly ask if they need support to get it done.

[Output Format]:

Provide a "Subject Line Option" followed by the "Email Body".



Modifier Stack

- **Task:** Reviewing a long article or industry update before a meeting.
- You mix and match your preferred shortcuts based on how much time you have.
- **User Interaction:**
/tldr /checklist /casual [Paste the 5-page article about new market trends]
- **How the System Compiles it:**

```
←!— System Injected: /tldr module →  
[INSTRUCTION]: Summarize the text into a  
maximum of 3 key takeaways.
```

```
←!— System Injected: /checklist module →  
[CONSTRAINT]: Convert those takeaways into  
immediate action items for our team.
```

```
←!— System Injected: /casual module →  
[TONE]: Write it in a relaxed,  
conversational style as if explaining it to  
a peer over coffee.
```

```
←!— User Input →  
[INPUT]: [5-page article text...]
```



God-Mode [Single-Line Method]

- **Task: Instant formatting of raw data during a busy workday.**
 - You don't have time to write paragraphs of instructions; you just want to pass a quick, dense string of commands to clean up messy text.
- **The Single-Line Command Pattern:**

```
/ROLE: Copyeditor  
/TASK: Fix grammar and typos  
/FORMAT: Bulleted list of corrections only :: I is writing this  
proposal for the client and they needs it by tomorrow morning.
```

↓ delimiter



Checkpoint

- Recap the definition of **Context Window**

Maximum number of combined **input, output, contextual information** tokens.

- We have covered $\text{Context}_{\text{input}}$, $\text{Context}_{\text{output}}$
- Context for contextual information?



Skill

- A skill is a **structured set** of instructions (packaged in a folder), that teaches an AI system how to perform specific tasks i.e. conducting structured research, creating standardized documents, managing multi-step processes.
- Skills allow you to define them once and reuse them consistently.
- Skills can also integrate with tools and systems, transforming raw capabilities into reliable, optimized workflows.



Design Principles

- The Skill folder comprises:
 - SKILL.md (required): Main instruction file (.md), including YAML frontmatter
 - scripts/ (optional): Executable / automation code (.py / .sh)
 - references/ (optional): Supporting documentation files
 - assets/ (optional): Resources like templates, fonts, or icons used in outputs
- The core design principles often consists of:
 1. Progressive Disclosure
 2. Composability
 3. Portability

```
---
name: job-quiz-generator
description: Generates quizzes for job seekers
trigger: when user asks for quiz or assessment
version: 1.0
---
```

```
your-skill-name/
├── SKILL.md           # Required - main skill file
├── scripts/          # Optional - executable code
│   ├── process_data.py # Example
│   └── validate.sh # Example
├── references/       # Optional - documentation
│   ├── api-guide.md # Example
│   └── examples/ # Example
└── assets/           # Optional - templates, etc.
    └── report-template.md # Example
```

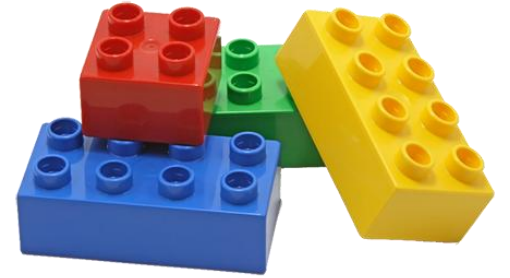


Progressive Disclosure

- Progressive Disclosure is a design principle where you only **load what's needed** and **hide the rest** until it's relevant.
- In terms of Skills, they follow a three-level loading structure:
 - **Level 1** (YAML frontmatter): Always loaded; signals when a skill should be used without loading every skill into the context
 - **Level 2** (SKILL.md body): Loaded when relevant; contains full instructions
 - **Level 3** (Linked files): Accessed on demand; includes supporting resources within the skill folder
- This layered approach reduces token usage while preserving specialised functionality.



Composability



- Composability means designing things so they can be **combined**, **reused**, and **work together** without breaking each other.
- The system can use multiple skills at the same time, so each skill should be designed to work with others rather than assuming it operates alone.



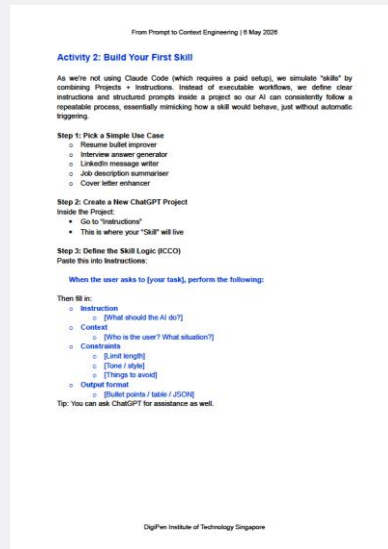
Portability

- Portability means your skill / system can be moved and used in different environments without breaking or needing major changes.
- Create them once and they can run consistently across different platforms.
- A good skill should work across:
 - Different AI models (GPT, Claude, etc.)
 - Different platforms (app, API, internal tools)
 - Different use cases (slight variations of the same task)



Activity #2 Skill Building

- Objective: Simulate skills using **Projects + Instructions**
- Duration: **30 minute**
- Material:



Testing and Iteration

- Skills can be tested at different level of thoroughness:
- Manual testing: Run queries directly and observe behavior. Fast iteration, no setup required.
- Scripted testing: Automate test cases for repeatability across changes.
- Programmatic testing via skills API: Design evaluation scenarios to systematically validate defined test cases



Testing Approach

1. Triggering tests
2. Functional tests
3. Performance comparison



Triggering Tests

- **Objective:** Ensure the skill is **invoked at the correct time**, similar to verifying whether a function or service is called under the right conditions.
- You're testing whether the system correctly detects when this skill should run, while also ensuring it avoids false positives.
- **Test Criteria**
 -  Triggers on clear, direct requests
 -  Triggers on variations or paraphrased inputs
 -  Does NOT trigger on unrelated queries



Example: Triggering Tests

Should Trigger	Should NOT Trigger
• “Set up a new project workspace” • “Create a project for my assignment” • “Initialize a project structure for Q4 planning”	• “What’s the weather today?” • “Help me debug Python code” • “Generate a spreadsheet”



Functional Tests

- **Objective:** Verify that the skill produces correct, complete, and reliable outputs, similar to unit or integration testing in software systems.
- You're testing whether the logic executes correctly, the outputs are valid and structured, external calls succeed, and edge cases are handled.
- **Test Criteria**
 -  Valid outputs are generated
 -  External API calls succeed
 -  Errors are handled gracefully
 -  Edge cases are covered



Example: Functional Tests

- **Test Case:** Create a project with 5 tasks
- **Given:** Project name: "Q4 Planning"
 - Input: 5 task descriptions
- **When:** Skill executes workflow
- **Expected Output:**
 - Project is created successfully
 - 5 tasks are generated with correct attributes
 - Tasks are linked to the project
 - No API or execution errors



Performance Comparison

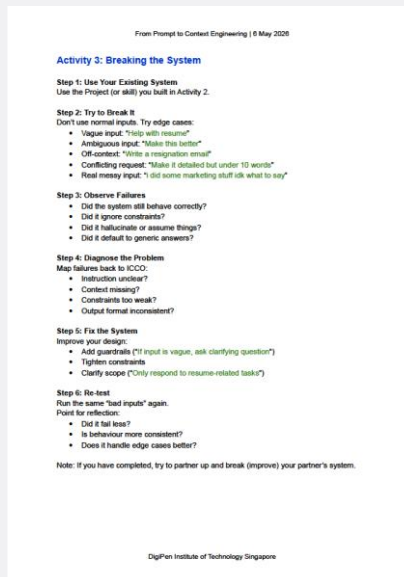
- **Objective:** Demonstrate that using the skill improves:
 - Efficiency, Reliability, Resource usage
- You're testing whether the skill reduces interaction overhead, minimises failures, and optimizes token and compute usage.

Automated	Manual
• Workflow executes automatically • Only ~2 clarification questions • 0 failed API calls • ~6,000 tokens consumed	• User needs to provide instructions step-by-step • ~15 back-and-forth messages • 3 failed API calls (retry needed) • ~12,000 tokens consumed



Activity #3 Breaking the System

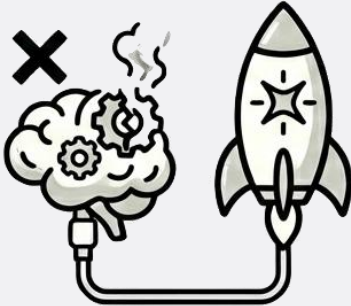
- Objective: Failure Testing
- Duration: **30 minute**
- Material:



The #1 Rule

Never Outsource Your Judgement

OUTSOURCED JUDGEMENT (Risk)



Risky outsourcing of
core decision-making.

UNCERTAINTY (If you can't tell...)



If accuracy is unclear,
don't rely on it.

NOT READY (Then you aren't ready...)



Uninformed use leads
to errors. Not ready.

HUMAN + AI (The Right Way)



Master your subject
Verify, then amplify.

References

- [Structured Output for Beginners: 3 Must-Know Prompting Tips](#)
- [The Complete Guide to Building Skills for Claude](#)

